

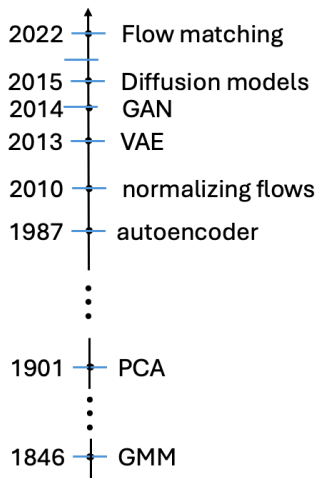
COMM114: Generative AI - Flow Matching (part II)

Guoqiang Zhang

September 2025

Flow Matching

Flow matching (FM) was invented in 2022 as a generalization of normalizing flows.



Recap from last lecture

Training a velocity field:

- **Input:** data samples $\{\mathbf{x}_i \in \mathbb{R}^d\}_{i=1}^n$.
- **Assumption:** Assume $\{\mathbf{x}_i\}_{i=1}^n$ are drawn from the data distribution $p_{data}(\cdot)$.
- Instead of learning the flow $\{\mathbf{X}_t^\theta \in \mathbb{R}^d | t \in [0, 1]\}$ directly, we learn a so-called **velocity field** $\mathbf{u}_t^\theta(\mathbf{X}_t^\theta)$ over $t \in [0, 1]$ from the dataset $\{\mathbf{x}_i \in \mathbb{R}^d\}_{i=1}^n$.
- The velocity field $\mathbf{u}_t^\theta(\mathbf{X}_t^\theta)$ is related with the flow $\mathbf{X}_t^\theta$ via an ordinary differential equation (ODE):

$$\frac{d}{dt}\mathbf{X}_t^\theta = \mathbf{u}_t^\theta(\mathbf{X}_t^\theta) \quad \text{s. t. } \mathbf{X}_{t=0}^\theta \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \quad [\text{ODE}]$$

- Once the velocity field $\mathbf{u}_t^\theta(\cdot)$ is obtained, the flow $\{\mathbf{X}_t^\theta | t \in [0, 1]\}$ is the solution of the ODE.

Recap from last lecture

Training of velocity field

- **Input:** data samples $\{\mathbf{x}_i \in \mathbb{R}^d \sim p_{data}(\mathbf{x})\}_{i=1}^n$.
- Let PDF of $\mathbf{X}_0$ be $p_{init} = \mathcal{N}(\mathbf{0}, \mathbf{I})$ (i.e., $p_{\mathbf{X}_0}(\mathbf{x}_0) = p_{init}(\mathbf{x})$)
- Let PDF of $\mathbf{X}_1$ be $p_{data}(\mathbf{x})$ (i.e., $p_{\mathbf{X}_1}(\mathbf{x}_1) = p_{data}(\mathbf{x})$)
- Define $\mathbf{X}_t = \alpha_t \mathbf{X}_0 + \beta_t \mathbf{X}_1$ where α_t and β_t satisfy:
 - α_t and β_t are scalar differentiable functions of $t \in [0, 1]$.
 - When $t = 0$, $\alpha_{t=0} = 1$ and $\beta_{t=0} = 0$.
 - When $t = 1$, $\alpha_{t=1} = 0$ and $\beta_{t=1} = 1$.
- Objective function for $\mathbf{u}_t^\theta(\cdot)$:

$$\begin{aligned} & \arg \min_{\theta} \mathbb{E}_{\mathbf{X}_1, \mathbf{X}_0} \left\| \mathbf{u}_t^\theta(\mathbf{X}_t) - \frac{d}{dt} \mathbf{X}_t \right\|^2 \\ &= \arg \min_{\theta} \mathbb{E}_{\mathbf{X}_1, \mathbf{X}_0} \left\| \mathbf{u}_t^\theta(\mathbf{X}_t) - \frac{d\alpha_t}{dt} \mathbf{X}_0 - \frac{d\beta_t}{dt} \mathbf{X}_1 \right\|^2 \end{aligned} \quad (1)$$

Recap from last lecture

Algorithm 1 Sampling from an FM model with Euler method

- 1: Input: velocity field $\mathbf{u}_t^\theta$ (a DNN model), number of steps k
 - 2: Set $t = 0$
 - 3: Set stepsize $h = \frac{1}{k}$
 - 4: Draw a sample $\mathbf{x}_0 \sim p_{init} = \mathcal{N}(0, \mathbf{I})$
 - 5: **for** $i = 1, \dots, k$ **do**
 - 6: $\mathbf{x}_{t+h} = \mathbf{x}_t + h\mathbf{u}_t^\theta(\mathbf{x}_t)$
 - 7: Set $t \leftarrow t + h$
 - 8: **end for**
 - 9: return $\mathbf{x}_{t=1}$
-

Remark 1: In principle, any ODE solver can be applied to perform sampling for an FM model.

Objective of this lecture

- Introduce advanced ODE solvers
- Learn a student ODE solver from a teacher ODE solver.

Heun's method

- Heun's method is a 2nd order ODE solver.
- Pros:
 - Higher accuracy than Euler's method due to its second-order convergence
 - Easy implementation
 - A good balance between computational simplicity and improved performance when solving the ODE.
- Cons:
 - Increased computational effort compared to Euler's method because it requires two function evaluations per step.
 - Potential error accumulation.

Heun's method (cont.)

Algorithm 2 Sampling from an FM model with Heun's method

- 1: Input: velocity field $\mathbf{u}_t^\theta$ (a DNN model), number of steps k
 - 2: Set $t = 0$
 - 3: Set stepsize $h = \frac{1}{k}$
 - 4: Draw a sample $\mathbf{x}_0 \sim p_{init} = \mathcal{N}(0, \mathbf{I})$
 - 5: **for** $i = 1, \dots, k$ **do**
 - 6: $\tilde{\mathbf{x}}_{t+h} = \mathbf{x}_t + h\mathbf{u}_t^\theta(\mathbf{x}_t)$
 - 7: $\mathbf{x}_{t+h} = \mathbf{x}_t + \frac{h}{2} (\mathbf{u}_t^\theta(\mathbf{x}_t) + \mathbf{u}_{t+h}^\theta(\tilde{\mathbf{x}}_{t+h}))$
 - 8: Set $t \leftarrow t + h$
 - 9: **end for**
 - 10: return $\mathbf{x}_{t=1}$
-

Remark: At time t , $\mathbf{x}_{t+h}$ is updated by accessing the DNN model two times, one at t and the other at $t + h$. By doing so, the approximation error introduced in Heun's method should be smaller than that of the Euler method.

Midpoint method

- The midpoint method offers improved accuracy over Euler's method for solving differential equations by using a midpoint value.
- Pros:
 - The midpoint method can provide a more accurate estimate than the simpler Euler method.
- Cons:
 - Increased computational effort compared to Euler's method because it requires two function evaluations per step.

Midpoint method (cont.)

Algorithm 3 Sampling from an FM model with Midpoint method

- 1: Input: velocity field $\mathbf{u}_t^\theta$ (a DNN model), number of steps k
 - 2: Set $t = 0$
 - 3: Set stepsize $h = \frac{1}{k}$
 - 4: Draw a sample $\mathbf{x}_0 \sim p_{init} = \mathcal{N}(0, \mathbf{I})$
 - 5: **for** $i = 1, \dots, k$ **do**
 - 6: $\mathbf{x}_{t+h} = \mathbf{x}_t + h\mathbf{u}_{t+\frac{h}{2}}^\theta \left(\mathbf{x}_t + \frac{h}{2}\mathbf{u}_t^\theta(\mathbf{x}_t) \right)$
 - 7: Set $t \leftarrow t + h$
 - 8: **end for**
 - 9: return $\mathbf{x}_{t=1}$
-

Two-step Adams–Bashforth

Algorithm 4 Sampling from an FM model with Two-step Adams-Bashforth method.

- 1: Input: velocity field $\mathbf{u}_t^\theta$ (a DNN model), number of steps k
 - 2: Set $t = 0$
 - 3: Set stepsize $h = \frac{1}{k}$
 - 4: Draw a sample $\mathbf{x}_0 \sim p_{init} = \mathcal{N}(0, \mathbf{I})$
 - 5: Set $\mathbf{x}_h = \mathbf{x}_0 + h\mathbf{u}_0^\theta(\mathbf{x}_0)$
 - 6: $t \leftarrow t + h$
 - 7: **for** $i = 1, \dots, k - 1$ **do**
 - 8: $\mathbf{x}_{t+h} = \mathbf{x}_t + \frac{3h}{2}\mathbf{u}_t^\theta(\mathbf{x}_t) - \frac{h}{2}\mathbf{u}_{t-h}^\theta(\mathbf{x}_{t-h})$
 - 9: Set $t \leftarrow t + h$
 - 10: **end for**
 - 11: return $\mathbf{x}_{t=1}$
-

Remark: see [website link](#) for details of Two-step Adams–Bashforth.

Learn a student ODE solver from a Teacher

Prior knowledge:

- In existing ODE solvers for a FM model, the coefficients of the ODE solvers are pre-determined.
- Typically, the step h needs to be sufficiently small to allow for high-quality data generation.
- Small h corresponds to large neural function evaluations (NFEs).

Remark: NFE refers to the number of times the FM model is accessed for generating a new data sample.

Learn a student ODE solver from a Teacher (cont.)

- Objective: To learn a student ODE solver which is able to perform fast sampling while maintaining the same generated data quality as the teacher ODE solver.
- Fast sampling implies only small NFEs are required in the sampling process when solving the ODEs.
- In the reference paper, a technique named *improved integration approximation* (IIA) is proposed to allow for learning a student ODE solver.

Reference: Guoqiang Zhang, Kenta Niwa, and W. Bastiaan Kleijn, "On Accelerating Diffusion-Based Sampling Processes via Improved Integration approximation", ICLR, 2024.

1st-Order Student ODE solver

- Suppose a 1st-order student ODE solver with learned coefficients $\{\beta_j\}_{j=1}^k$ over a set of coarse timesteps $\{t_j\}_{j=0}^k$ is available.

Algorithm 5 Sampling with a learned student ODE solver.

- 1: Input: velocity field $\mathbf{u}_t^\theta$ (a DNN model)
 - 2: Input: timesteps $\{t_j\}_{j=0}^k$ where $t_0 = 0$ and $t_k = 1$.
 - 3: Draw a sample $\mathbf{x}_0 \sim p_{init} = \mathcal{N}(0, \mathbf{I})$
 - 4: **for** $j = 0, \dots, k - 1$ **do**
 - 5: $\mathbf{x}_{t_{j+1}} = \mathbf{x}_{t_j} + \beta_j \mathbf{u}_{t_j}^\theta(\mathbf{x}_{t_j})$
 - 6: **end for**
 - 7: return $\mathbf{x}_{t_k=1}$
-

Remark: We replace the coefficient h of the Euler method with the learned coefficient β_i at the i th sampling step.

1st-Order Student ODE solver (cont.)

- Learning the coefficients $\{\beta_j\}_{j=0}^{k-1}$ sequentially from $j = 0$ until $j = k - 1$.
- Each coefficient is learned by minimizing a quadratic function

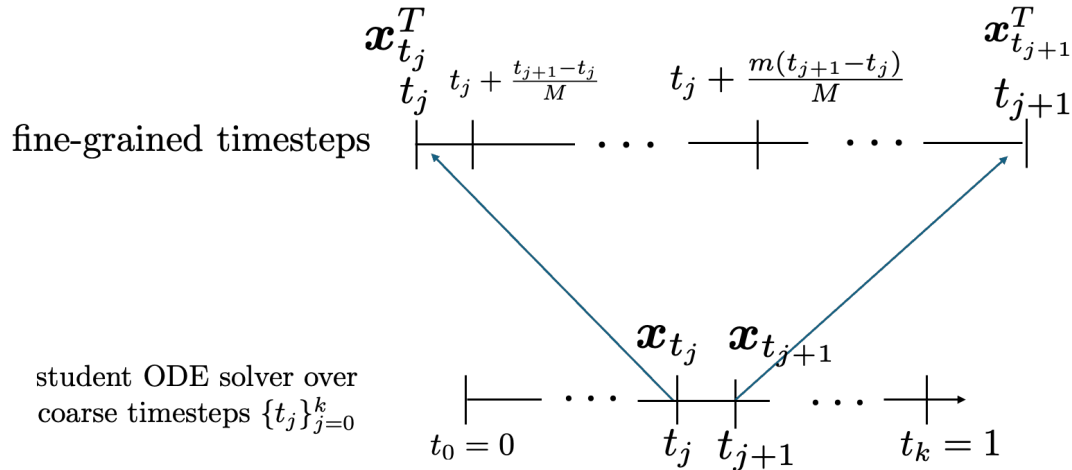
$$\beta_j^* = \arg \min \mathbb{E}_{\mathbf{x}_{t_0} \sim p_{init} = \mathcal{N}(0, \mathbf{I})} \left\| \underbrace{\mathbf{x}_{t_j} + \beta_j \mathbf{u}_{t_j}^{\theta}(\mathbf{x}_{t_j})}_{\text{Euler step}} - \mathbf{x}_{t_{j+1}}^T \right\|^2$$

where

$$\mathbf{x}_{t_{j+1}}^T = \text{Teacher-ODE-solver} \left(\mathbf{x}_{t_j}^T, \left\{ t_j + \frac{m}{M} (t_{j+1} - t_j) \right\}_{m=0}^M \right)$$

- The teacher ODE solvers produces a much more accurate state $\mathbf{x}_{t_{j+1}}^T$ by solving the ODE for M steps within each time-slot $[t_j, t_{j+1}]$.

1st-Order Student ODE solver (cont.)



2nd-Order Student ODE solver

- Suppose a 2nd-order student ODE solver with learned coefficients $\{\beta_0\} \cup \{\beta_j, \gamma_j\}_{j=1}^{k-1}$ over a set of coarse timesteps $\{t_j\}_{j=0}^k$ is available.

Algorithm 6 Sampling with a learned student ODE solver.

- 1: Input: velocity field $\mathbf{u}_t^\theta$ (a DNN model)
 - 2: Input: timesteps $\{t_j\}_{j=0}^k$ where $t_0 = 0$ and $t_k = 1$.
 - 3: Draw a sample $\mathbf{x}_0 \sim p_{init} = \mathcal{N}(0, \mathbf{I})$
 - 4: $\mathbf{x}_{t_1} = \mathbf{x}_{t_0} + \beta_0 \mathbf{u}_{t_0}^\theta(\mathbf{x}_{t_0})$
 - 5: **for** $j = 1, \dots, k-1$ **do**
 - 6: $\mathbf{x}_{t_{j+1}} = \mathbf{x}_{t_j} + \beta_j \mathbf{u}_{t_j}^\theta(\mathbf{x}_{t_j}) + \gamma_j \left[\mathbf{u}_{t_j}^\theta(\mathbf{x}_{t_j}) - \mathbf{u}_{t_{j-1}}^\theta(\mathbf{x}_{t_{j-1}}) \right]$
 - 7: **end for**
 - 8: return $\mathbf{x}_{t_k=1}$
-

Remark: At each time t_j , we use of $\mathbf{u}_{t_j}^\theta(\mathbf{x}_{t_j})$ and $\mathbf{u}_{t_{j-1}}^\theta(\mathbf{x}_{t_{j-1}})$ in computation of $\mathbf{x}_{t_{j+1}}$.

2nd-Order Student ODE solver (cont.)

- Learning the coefficients $\{\beta_0\} \cup \{\beta_j, \gamma_j\}_{j=1}^{k-1}$ sequentially from $j = 0$ until $j = k - 1$.
- Each pair of coefficients are learned by minimizing a quadratic function

$$\beta_j^*, \gamma_j^* = \arg \min \mathbb{E}_{\mathbf{x}_{t_0} \sim p_{init} = \mathcal{N}(0, \mathbf{I})} \left\| \mathbf{x}_{t_j} + \beta_j \mathbf{u}_{t_j}^{\theta}(\mathbf{x}_{t_j}) + \gamma_j \left[\mathbf{u}_{t_j}^{\theta}(\mathbf{x}_{t_j}) - \mathbf{u}_{t_{j-1}}^{\theta}(\mathbf{x}_{t_{j-1}}) \right] - \right.$$

where

$$\mathbf{x}_{t_{j+1}}^T = \text{Teacher-ODE-solver} \left(\mathbf{x}_{t_j}^T, \left\{ t_j + \frac{m}{M}(t_{j+1} - t_j) \right\}_{m=0}^M \right)$$

- The teacher ODE solvers produces a much more accurate state $\mathbf{x}_{t_{j+1}}^T$ by solving the ODE for M steps within each time-slot $[t_j, t_{j+1}]$.

A note on 2nd-Order Student ODE solver

- The 2nd-order student ODE solver can take a different form. For example, we could let

$$\mathbf{x}_{t_{j+1}} = \mathbf{x}_{t_j} + \beta_j \mathbf{u}_{t_j}^{\theta}(\mathbf{x}_{t_j}) + \gamma_j [\mathbf{x}_{t_j} - \mathbf{x}_{t_{j-1}}],$$

where

$$\beta_j^*, \gamma_j^* = \arg \min \mathbb{E}_{\mathbf{x}_{t_0} \sim p_{init} = \mathcal{N}(0, \mathbf{I})} \left\| \mathbf{x}_{t_j} + \beta_j \mathbf{u}_{t_j}^{\theta}(\mathbf{x}_{t_j}) + \gamma_j [\mathbf{x}_{t_j} - \mathbf{x}_{t_{j-1}}] - \mathbf{x}_{t_{j+1}}^T \right\|^2$$

where

$$\mathbf{x}_{t_{j+1}}^T = \text{Teacher-ODE-solver} \left(\mathbf{x}_{t_j}^T, \left\{ t_j + \frac{m}{M} (t_{j+1} - t_j) \right\}_{m=0}^M \right).$$

Remark: Selection of the particular format of the student ODE solver should depend on which one gives the lowest residual signal in the quadratic function.

Implementation Guidance

- In practice, one can solve the quadratic optimization problems by using a minibatch of samples, e.g., (100 samples).
- Once the coefficients for a student ODE solver are learned, they need to be stored in hard disk.
- When performing sampling, the coefficients can be loaded into the memory and used for generating new data samples.

Questions?